

IPC Experiment - Pipes & Named Pipes

Experiment 1: Unnamed Pipe (Parent to Child)

Concept

- pipe(fd) creates a unidirectional communication channel
- Works only between related processes (parent-child via fork)
- fd[0] = Read end, fd[1] = Write end
- Data flows: Parent writes to fd[1] --> Child reads from fd[0]

Common Mistakes to Avoid

- Not closing unused pipe ends (causes deadlock)
- Reading/writing from the wrong ends
- Not checking fork() return value

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2];
    pid_t pid;
    char write_msg[] = "Hello from parent!";
    char read_msg[100];

    if (pipe(fd) == -1) {
        perror("Pipe failed");
        exit(1);
    }

    pid = fork();
    if (pid < 0) {
        perror("Fork failed");
        exit(1);
    }

    // Parent Process
    if (pid > 0) {
        close(fd[0]); // Close read end
        write(fd[1], write_msg, strlen(write_msg) + 1);
        close(fd[1]);
    }
    // Child Process
    else {
        close(fd[1]); // Close write end
        read(fd[0], read_msg, sizeof(read_msg));
        printf("Child received: %s\n", read_msg);
        close(fd[0]);
    }
}
```

Compile & Run

```
gcc unnamed_pipe.c -o unnamed_pipe
./unnamed_pipe
```

Key Viva Points

- fd[0] is always the read end, fd[1] is always the write end
- Unnamed pipes only work between related processes
- Always close unused ends to avoid blocking

Experiment 2: Named Pipe / FIFO (file1 Sends, file2 Receives)

Concept

- mkfifo() creates a named pipe visible in the filesystem
- Works between completely unrelated processes
- file1.c = Sender (Writer), file2.c = Receiver (Reader)
- IMPORTANT: Run receiver FIRST, then sender - otherwise it blocks

file1.c - Sender (Writer)

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#include <string.h>

#define FIFO_NAME "myfifo"

int main() {
    int fd;
    char data[100];

    mkfifo(FIFO_NAME, 0666);          // Create FIFO
    fd = open(FIFO_NAME, O_WRONLY);   // Open for writing

    printf("Enter message: ");
    fgets(data, sizeof(data), stdin);

    write(fd, data, strlen(data) + 1);
    close(fd);

    return 0;
}
```

file2.c - Receiver (Reader)

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

#define FIFO_NAME "myfifo"

int main() {
    int fd;
    char data[100];

    fd = open(FIFO_NAME, O_RDONLY);    // Open for reading
    read(fd, data, sizeof(data));

    printf("Received from file1: %s", data);
    close(fd);

    return 0;
}
```

Compile & Run

```
gcc file1.c -o file1
gcc file2.c -o file2

./file2    # Run receiver FIRST
./file1    # Then run sender
```

Key Viva Points

- mkfifo() creates the named pipe as a file in the filesystem
- Named pipes persist in filesystem until manually deleted
- Works between unrelated processes - no fork() needed
- Both ends must open before data transfer can begin

Experiment 3: Named Pipe Reversed (file2 Sends, file1 Receives)

Roles are reversed - file2 becomes the sender, file1 becomes the receiver. The code is identical to Experiment 2 with only the print message changed.

file2.c - Sender (Writer) [Same as file1.c above]

- Use same sender code from Experiment 2
- Change printf label to: "Enter message (file2 sending):"

file1.c - Receiver (Reader) [Same as file2.c above]

- Use same receiver code from Experiment 2
- Change printf to: printf("Received from file2: %s", data);

Compile & Run

```
gcc file2.c -o file2
gcc file1.c -o file1

./file1    # Run receiver FIRST
./file2    # Then run sender
```

Quick Reference Summary

Feature	Unnamed Pipe	Named Pipe (FIFO)	System Call
Processes	Parent-Child only	Any processes	fork(), pipe()
Filesystem	No file created	File in filesystem	mkfifo()
Direction	Unidirectional	Unidirectional	read(), write()
Run Order	fork() handles it	Receiver runs first	open(), close()